Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе №1
Тема: «Проектирование комбинационных схем с использованием языка VHDL»

Выполнила У.В. Прокопюк

Проверил А.А. Воронов

Минск 2026

1.1 Постановка задачи

Цели:

1. Ознакомиться с САПР Xilinx Vivado и приобрести навыки работы с ним.
2. Изучить базовый синтаксис языка VHDL.
3. Приобрести навыки проектирования простейших комбинационных схем на языке VHDL.

1.2 Задание

В соответствии с полученным вариантом задания необходимо спроектировать комбинационное устройство на языке VHDL. В качестве исходных данных предоставляется схема устройства для реализации. В случае отсутствия табличного описания его работы необходимо выполнить анализ его схемотехники для понимания принципов его работы.

Устройство должно быть реализовано двумя способами:

1. С использованием исключительно логических операторов и параллельного оператора безусловного присваивания.
2. С использованием последовательных операторов за исключением оператора присваивания.

Первый вариант подразумевает запрет на использование других операторов языка VHDL.

Во втором варианте можно использовать любые другие операторы (в том числе и оператор присваивания), но при этом последовательные операторы также должны присутствовать в описании устройства.

После создания описания устройства необходимо разработать для него тестовый модуль, с целью проверки и демонстрации его работы. Тестовый модуль должен выполнять полное тестирование разработанного устройства, что для комбинационного устройства предполагает полный перебор входных значений.

Последним этапом является выполнение поведенческого моделирования устройства с использованием тестового модуля. Моделирование выполняется для всех вариантов описания устройства.

1.3 Выполнение задания

Вариант задания по которой необходимо спроектировать комбинационное устройство на языке VHDL. В качестве исходных данных предоставляется схема устройства для реализации на Рисунке 1.1.

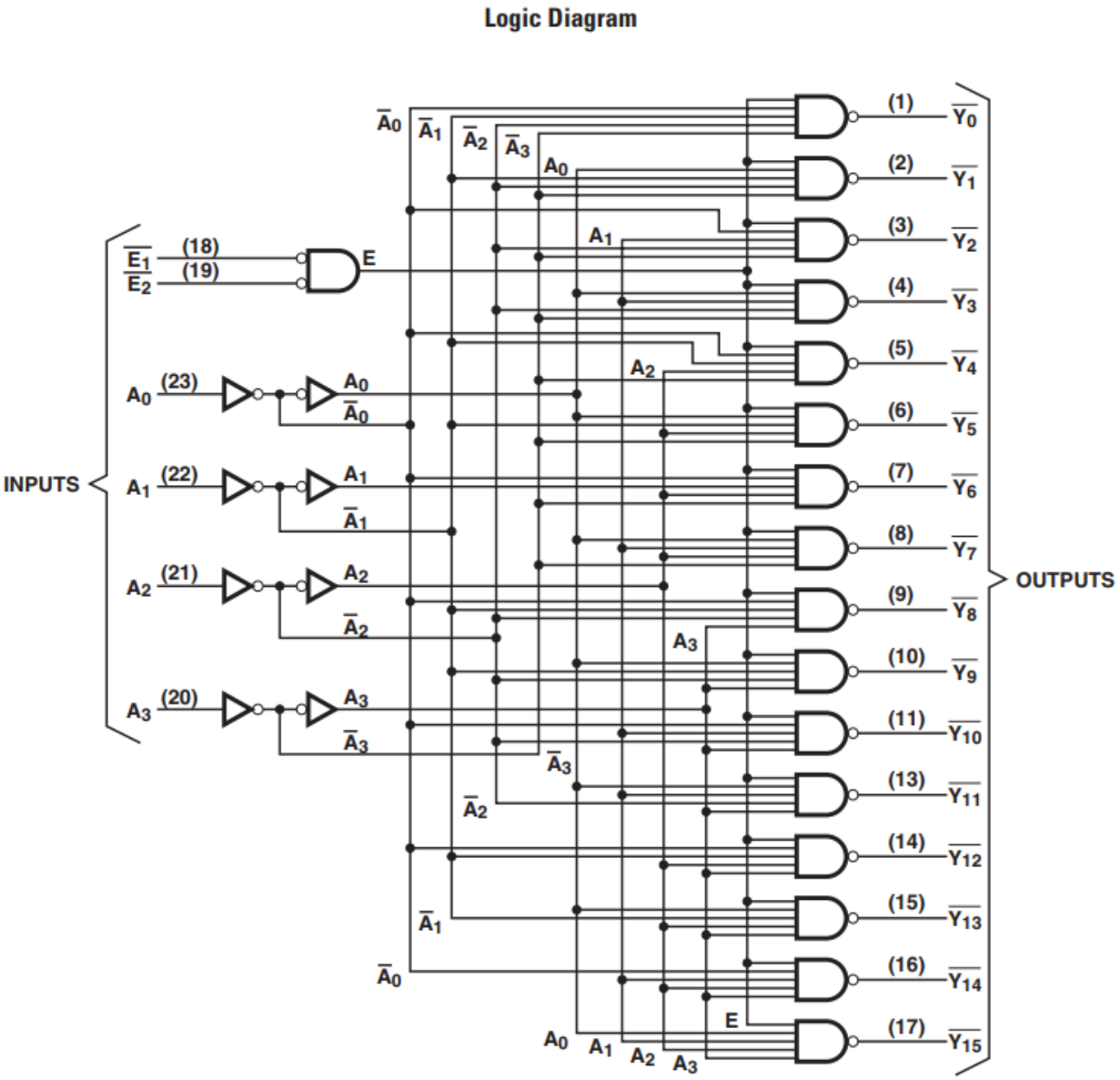


Рисунок 1.1 – 4-линейные до 16-линейных декодеров/демультиплексоров

Результат работы представлен в виде исходного кода программы на языке VHDL, а также результат значений на выходах в соответствии со входными значениями на рисунке 1.2.

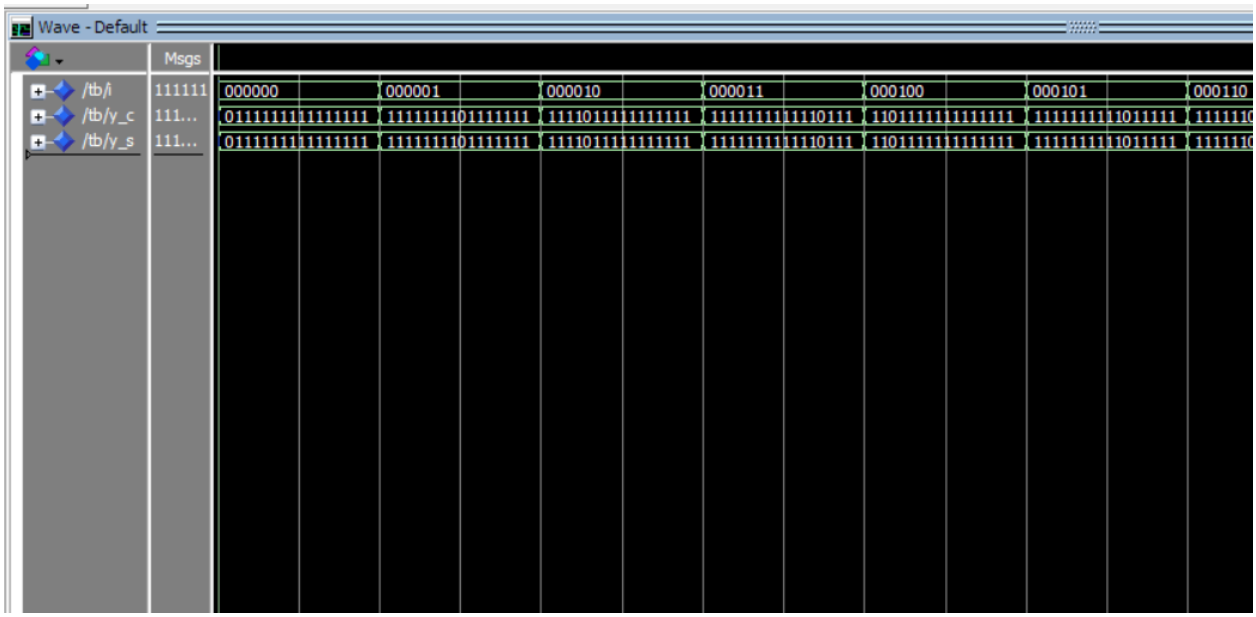


Рисунок 1.2 – Результат работы

Исходный код файлов:

Lab1.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity lab1 is
    port (
        i : in  std_logic_vector(1 to 6);    -- I1..I6
        y : out std_logic_vector(0 to 15)    -- Y0..Y15
    );
end entity;

-- ============================================================
-- Вариант 1: только not/and/or + concurrent присваивания
-- ============================================================
architecture concurrent_only of lab1 is
    -- общий узел схемы: A = 2и(не(I1), не(I2))
    signal a : std_logic;

    -- для читабельности вынесем инверсии
    signal n1, n2, n3, n4, n5, n6 : std_logic;
begin
    n1 <= not i(1);
    n2 <= not i(2);
    n3 <= not i(3);
    n4 <= not i(4);
    n5 <= not i(5);
    n6 <= not i(6);

    a  <= n1 and n2;  -- 2и(не(I1), не(I2))
```

```
        -- 5и-не(x1,x2,x3,x4,x5) = not(x1 and x2 and x3 and x4 and
x5)

        y(0)  <= not( a and n3 and n4 and n5 and n6 );
        y(1)  <= not( a and i(3) and n4 and n5 and n6 );
        y(2)  <= not( a and n3 and i(4) and n5 and n6 );
        y(3)  <= not( a and i(3) and i(4) and n5 and n6 );
        y(4)  <= not( a and n3 and n4 and i(5) and n6 );
        y(5)  <= not( a and i(3) and n4 and i(5) and n6 );
        y(6)  <= not( a and n3 and i(4) and i(5) and n6 );
        y(7)  <= not( a and i(3) and i(4) and i(5) and n6 );

        y(8)  <= not( a and n3 and n4 and n5 and i(6) );
        y(9)  <= not( a and i(3) and n4 and n5 and i(6) );
        y(10) <= not( a and n3 and i(4) and n5 and i(6) );
        y(11) <= not( a and i(3) and i(4) and n5 and i(6) );
        y(12) <= not( a and n3 and n4 and i(5) and i(6) );
        y(13) <= not( a and i(3) and n4 and i(5) and i(6) );
        y(14) <= not( a and n3 and i(4) and i(5) and i(6) );
        y(15) <= not( a and i(3) and i(4) and i(5) and i(6) );
    end architecture;

    -- ============================================================
    -- Вариант 2: последовательные операторы + ветвление (VHDL-93)
    -- ============================================================
    architecture sequential of lab1 is
    begin
        process(i)
            variable a : std_logic;
            variable n1, n2, n3, n4, n5, n6 : std_logic;
            variable yv : std_logic_vector(0 to 15);
        begin
            n1 := not i(1);
            n2 := not i(2);
            n3 := not i(3);
            n4 := not i(4);
            n5 := not i(5);
            n6 := not i(6);

            a  := n1 and n2;

            -- ветвление: если a=0, то во всех выражениях множитель
a=0,
            -- значит (a and ...) = 0 и NAND даёт '1' на всех
выходах.
            if a = '0' then
                yv := (others => '1');
            else
                yv(0)  := not( a and n3 and n4 and n5 and n6 );
                yv(1)  := not( a and i(3) and n4 and n5 and n6 );
                yv(2)  := not( a and n3 and i(4) and n5 and n6 );
```

```
                yv(3)  := not( a and i(3) and i(4) and n5 and n6
);
                yv(4)  := not( a and n3 and n4 and i(5) and n6 );
                yv(5)  := not( a and i(3) and n4 and i(5) and n6
);
                yv(6)  := not( a and n3 and i(4) and i(5) and n6
);
                yv(7)  := not( a and i(3) and i(4) and i(5) and n6
);

                yv(8)  := not( a and n3 and n4 and n5 and i(6) );
                yv(9)  := not( a and i(3) and n4 and n5 and i(6)
);
                yv(10) := not( a and n3 and i(4) and n5 and i(6)
);
                yv(11) := not( a and i(3) and i(4) and n5 and i(6)
);
                yv(12) := not( a and n3 and n4 and i(5) and i(6)
);
                yv(13) := not( a and i(3) and n4 and i(5) and i(6)
);
                yv(14) := not( a and n3 and i(4) and i(5) and i(6)
);
                yv(15) := not( a and i(3) and i(4) and i(5) and
i(6) );
            end if;

            y <= yv;
        end process;
    end architecture;

    tb.vhd
    library ieee;
    use ieee.std_logic_1164.all;
    use ieee.numeric_std.all;

    entity tb is
    end entity;

    architecture sim of tb is
        signal i : std_logic_vector(1 to 6) := (others => '0');
        signal y_c : std_logic_vector(0 to 15);
        signal y_s : std_logic_vector(0 to 15);
    begin
        u1: entity work.lab1(concurrent_only)
            port map ( i => i, y => y_c );

        u2: entity work.lab1(sequential)
            port map ( i => i, y => y_s );

        stim: process
            variable k  : integer;
```

```
        variable v  : std_logic_vector(5 downto 0);
    begin
        for k in 0 to 63 loop
            v := std_logic_vector(to_unsigned(k, 6));

            -- разложим v(5 downto 0) в i(1..6)
            -- пусть v(5)=I1 ... v(0)=I6
            i(1) <= v(5);
            i(2) <= v(4);
            i(3) <= v(3);
            i(4) <= v(2);
            i(5) <= v(1);
            i(6) <= v(0);

            wait for 10 ns;

            assert y_c = y_s
                report "Mismatch at k=" & integer'image(k)
                severity error;
        end loop;

        report "TB finished OK" severity note;
        wait;
    end process;
end architecture;
```

1.4 Заключение

В ходе выполнения лабораторной работы было освоено использование и закреплён базовый синтаксис языка VHDL. На основе заданной схемы разработано комбинационное устройство (4-линейные до 16-линейных декодеров/демультиплексоров) и выполнено его описание двумя способами: с применением только логических операторов not/and/or и параллельных безусловных присваиваний, и с использованием последовательного описания в процессе с ветвлением. Для проверки работоспособности создан тестовый модуль, обеспечивающий полный перебор всех 16 комбинаций входных сигналов и сравнение результатов двух реализаций. По итогам поведенческого моделирования подтверждено совпадение выходов обеих архитектур и корректность работы устройства, что демонстрирует правильность выполненного проектирования и тестирования.